

Dossier technique — Scrabble

Ce document décrit le projet de bout en bout. On y explique d'abord ce qu'on a construit, puis on suit le déroulement réel d'une partie en faisant intervenir chaque classe au moment où elle joue son rôle. Le but : pouvoir répondre à n'importe quelle question sur un fichier ou une méthode en sachant **pourquoi** elle existe et **comment** elle s'inscrit dans la mécanique générale, pas seulement **ce qu'elle fait**.

1. De quoi on est parti

Le projet est une implémentation jouable du Scrabble pour 2 à 4 joueurs sur le même écran. On a séparé l'application en deux briques qui ne se connaissent que par un contrat JSON :

- un **backend Java** qui gère toute la mécanique (plateau, sac, validation, score, historique, dictionnaire) et expose une API HTTP ;
- un **frontend React** qui affiche le plateau, les chevalets, l'historique, et collecte les actions du joueur.

Le backend ne sait pas qu'un navigateur l'appelle. Le frontend ne connaît rien à Java. Le seul lien est l'API REST exposée sur <http://localhost:8080/api/>.... Cela rend la logique métier testable sans HTTP, et le frontend remplaçable sans toucher au reste.

L'**état complet vit côté serveur**. Le frontend ne stocke que des intentions transitoires (tuiles posées localement avant validation, mode courant, direction sélectionnée). Recharger la page, c'est juste rappeler </api/game/state> et reprendre à l'identique.

2. Les règles que le programme doit savoir faire jouer

Toutes les règles ci-dessous sont implémentées et **vérifiées côté backend** : le frontend ne fait aucune validation, ce qui empêche toute triche par bidouillage du client.

Le plateau est une grille 15×15. Quelques cases portent des **primes** : lettre comptée double (LD) ou triple (LT), mot compté double (MD) ou triple (MT), et la case centrale (7,7) qui est la case de **départ** et qui compte comme un mot double sur le tout premier coup. Une prime n'est consommée qu'une seule fois — la première fois qu'une lettre est posée dessus. Les lettres déjà sur le plateau ne déclenchent plus rien.

Le sac contient 102 tuiles avec la composition standard du Scrabble français : 15 E, 9 A, 8 I, 6 N/O/R/S/T/U, 5 L, 3 D/M, 2 B/C/F/G/H/P/V, 1 J/K/Q/W/X/Y/Z, et 2 jokers (?). Chaque tuile a une valeur en points (de 0 pour le joker à 10 pour W/X/Y/Z/K). Au début, chaque joueur tire 7 tuiles au hasard.

À chaque tour, le joueur courant a **trois choix exclusifs** : poser un mot, échanger des tuiles (si le sac contient ≥ 7 tuiles), ou passer son tour. Poser un mot donne des points et reset le compteur de passes. Échanger ou passer ne donne aucun point et incrémente le compteur ; quand ce compteur atteint **2 × le nombre de joueurs**, la partie s'arrête.

Quand un joueur pose un mot, il dépose une ou plusieurs tuiles **alignées** (toutes sur la même ligne ou la même colonne), et **contiguës** (le mot principal n'a pas de trou). Sauf au tout premier coup (qui doit obligatoirement passer par le centre), le coup doit **toucher** au moins une tuile déjà sur le plateau. Tous les

mots formés — le mot principal mais aussi tous les mots transversaux créés — doivent être dans le dictionnaire.

Le score d'un coup est calculé mot par mot : on additionne les valeurs des lettres, on applique les multiplicateurs LD/LT aux nouvelles tuiles posées sur ces cases, puis on multiplie le sous-total par tous les multiplicateurs MD/MT (et DEPART) couverts par les nouvelles tuiles. Si le joueur pose ses 7 tuiles d'un coup, on ajoute **+50 points** : le « scrabble ».

Un **joker** vaut 0 point. Au moment de le poser, le joueur choisit la lettre qu'il représente. Cette face est figée : sur le plateau, c'est désormais une lettre normale qui vaut 0 point.

Deux scénarios mettent fin à la partie. Premier : un joueur vide son chevalet alors que le sac est déjà vide ; tous les autres joueurs perdent les points encore présents sur leur chevalet, et celui qui a fini reçoit en bonus la somme totale de ces pénalités. Deuxième : trop de passes consécutives sont enchaînées ; tous les joueurs perdent leurs points restants, sans bonus pour personne. Dans les deux cas, le joueur au plus haut score remporte la partie.

3. Découpage en couches et fil rouge

Le backend est organisé en couches strictement orientées : **model** (entités du jeu) → **service** (règles métier) → **controller** (orchestration) → **api** (HTTP/JSON) → **view** (point d'entrée). Une couche basse n'importe **jamais** une couche haute. C'est ce qui permet, par exemple, d'écrire un test du **ValideurCoup** sans aucun serveur HTTP, ou de remplacer le serveur HTTP par une CLI sans toucher à la logique du jeu.

Le frontend est plus simple : un composant **App.jsx** qui porte tout l'état UI, un fichier **api.js** qui fait les appels HTTP, et **styles.css** pour l'apparence.

Pour ne pas tomber dans un catalogue de classes, on va dérouler le projet selon **le déroulement réel d'une partie**. Chaque classe sera introduite quand elle entre en scène.

```
scrabble_v4/
├── src/                                ← Code Java (backend)
│   ├── view/                          Point d'entrée (Main)
│   ├── api/                           Serveur HTTP, parsing/sérialisation JSON
│   ├── controller/                    Orchestration des requêtes
│   ├── service/                       Logique métier (validation, score, dictionnaire)
│   └── model/                         Entités du domaine
├── frontend-react/                    Code React (frontend)
│   └── src/                           main.jsx, App.jsx, api.js, styles.css
├── dictionnaire/                      Liste de mots + scripts de téléchargement
├── run-backend.bat                     Compile + lance le backend (Windows)
└── run-backend.sh                      Compile + lance le backend (Unix)
```

4. Au démarrage : ce qui s'instancie

Quand l'utilisateur lance **run-backend.bat** (ou **.sh**), le script compile tous les **.java** sous **src/** dans **out/** puis lance **java -cp out view.Main**. La classe **Main** est volontairement minimale : elle crée un

`ScrabbleHttpServer` sur le port 8080 et l'appelle à `start()`. Tout le câblage est dans le serveur HTTP, pas dans `Main`.

Le constructeur de `ScrabbleHttpServer` fait trois choses qui méritent d'être détaillées. Premièrement, il **charge le dictionnaire**. La méthode privée `chargerDictionnaire()` essaie d'instancier un `ServiceDictionnaireFichier` pointant vers `dictionnaire/mots.txt`. Si le fichier existe, le service en lit toutes les lignes via `Files.readAllLines(...)`, et **normalise** chaque mot avant de le stocker dans un `HashSet<String>`. La normalisation est la même que celle appliquée plus tard aux mots à valider : `trim()`, passage en majuscules avec `Locale.ROOT` (pour éviter les bizarreries comme le `i` turc), décomposition NFD pour séparer les lettres des accents, suppression des `\p{M}` (les marques diacritiques), puis suppression de tout ce qui n'est pas A-Z. Concrètement, `île`, `Ile`, `île` deviennent tous `ILE`. Si le fichier n'existe pas, on retombe sur `ServiceDictionnaireToujoursValide` qui accepte tout, et on logge un avertissement. Cette stratégie permet de jouer même sans dictionnaire (utile pour montrer la mécanique pure pendant le développement) tout en activant la vraie validation dès qu'on fournit un fichier.

Le `ServiceDictionnaire` est défini comme une interface à **une seule méthode**, `boolean estValide(String mot)`. C'est volontairement minimal : tant qu'on a une source quelconque (fichier, base de données, API distante, dictionnaire en mémoire pour les tests), on peut la brancher sans toucher au reste. Cette interface sera consommée plus tard par le validateur de coups.

Deuxièmement, `ScrabbleHttpServer` instancie un `ContrôleurPartie`, en lui passant le service de dictionnaire qu'il vient de charger. Le contrôleur, à son tour, instancie un `PartieService(dictionnaire)`. Le `PartieService` construit alors un `ValideurCoup(dictionnaire)` et un `ServiceScore()`. Cette chaîne d'instanciation forme l'**injection de dépendance** : le dictionnaire est choisi en haut, et descend de couche en couche jusqu'au validateur qui s'en sert.

Troisièmement, le serveur déclare ses sept routes — `/api/health`, `/api/game/state`, `/api/game/start`, `/api/game/reset`, `/api/game/play`, `/api/game/pass`, `/api/game/exchange` — chacune liée à une méthode privée. Toutes passent par un même wrapper interne, `JsonHandler`, qui fait quatre choses : poser les en-têtes CORS et `Content-Type: application/json; charset=utf-8`, court-circuiter les requêtes `OPTIONS` (preflights CORS) avec un 204, lire et parser le corps JSON via `Json.parse(...)`, et capturer les exceptions pour les convertir en réponses HTTP. Une `IllegalArgumentException` ou `IllegalStateException` devient un HTTP 400 avec un corps `{"error": "..."} ;` toute autre exception devient un 500. Cela libère chaque route de la gestion d'erreur : elles peuvent juste lever une exception métier descriptive et se reposer sur ce mécanisme.

Le `Json.java` mérite un mot. C'est un parser/serializer JSON écrit à la main, ≈ 250 lignes, qui couvre exactement ce dont on a besoin : objets, listes, chaînes (avec échappements `\`, `\\`, `\b`, `\f`, `\n`, `\r`, `\t`, `\uXXXX`), nombres (entiers ou décimaux), booléens, null. Les types de retour côté Java sont `Map<String, Object>`, `List<Object>`, `String`, `Long`, `Double`, `Boolean`, ou `null`. Pourquoi pas Jackson ou Gson ? Parce qu'on voulait conserver « zéro dépendance externe » : pas de Maven, pas de Gradle, juste un `javac` et un `java`. La compilation est instantanée et le projet est trivialement reproductible.

Enfin, le serveur configure un `Executors.newCachedThreadPool()` : chaque requête est traitée dans un thread du pool, ce qui rend le serveur naturellement multi-clients (même si en pratique notre projet n'en utilise qu'un à la fois).

Une fois tout ça câblé, `start()` lance `HttpServer` et le backend log `Dictionnaire chargé : N mots...` puis `Backend Scrabble lancé sur http://localhost:8080`. Il attend les requêtes.

5. Représenter une partie en mémoire

Avant qu'une vraie partie commence, on a besoin d'un vocabulaire d'objets. Tout est dans `src/model/`, et aucune de ces classes ne sait parler HTTP : ce sont des entités du domaine, pures.

Le centre de gravité est `Partie`. Cette classe est **mutable** parce qu'une partie évolue dans le temps. Elle contient un `Plateau`, un `SacTuiles`, une liste de `Joueur`, l'index du joueur courant, le compteur de passes consécutives (`passesConsecutifs`), un booléen `terminee`, le dernier message + les derniers points + les derniers mots (pour que le frontend affiche la boîte « Dernière action »), et la liste de l'**historique** (on y revient plus loin). Toutes les méthodes qui mutent l'état sont publiques mais utilisées **exclusivement** par `PartieService` ; le contrôleur n'y touche pas directement.

Un `Plateau` est une matrice 15×15 de `CasePlateau`. À sa construction, il consulte `DispositionPlateau.primeA(ligne, colonne)` pour fixer la prime de chaque case. Cette classe utilitaire — qui n'est pas instanciable — déclare statiquement les coordonnées des primes en exploitant la **symétrie centrale et axiale** du plateau : pour chaque coordonnée déclarée, la classe ajoute automatiquement ses 4 (ou 8) symétriques, ce qui réduit le code à un quart du plateau. La case centrale (7,7) est traitée à part avec `Prime.DEPART`. Les `Set<Long>` qui stockent les coordonnées encodent (`ligne, colonne`) en un long 64 bits — un détail d'optimisation qui évite d'instancier des `Position` pour la table de lookup. Une `CasePlateau` connaît sa prime (figée à la construction) et porte une `Tuile` mutable (`null` quand vide).

Une `Tuile` est volontairement immuable : elle a un identifiant, une lettre, une valeur en points. L'identifiant est généré par un `AtomicInteger` statique (`COMPTEUR`) : `T1`, `T2`, `T3`, etc. **Cet ID est crucial** : c'est lui qui permet au frontend de désigner précisément une tuile dans son chevalet, même quand le joueur a deux `E` ou deux `S`. Sans ID stable, on ne pourrait pas distinguer « le E que je viens de tirer » de « le E que j'avais déjà ». Une tuile joker est repérée par le caractère `?` ; sa méthode `estJoker()` est utilisée à plusieurs endroits, notamment au moment de la pose pour vérifier qu'une face a été choisie.

Le `SacTuiles` est une `Deque<Tuile>` mélangée. À sa construction, il instancie 9 A à 1 point, 2 B à 3 points, et ainsi de suite jusqu'aux 2 jokers à 0 point, puis appelle `Collections.shuffle(...)` avant de tout empiler. La méthode `piocher()` retire la tuile en haut ; `piocher(int n)` pioche jusqu'à `n` tuiles ; `remettre(List<Tuile>)` ajoute des tuiles au sac et **re-mélange tout** — important pour qu'un joueur qui échange juste après un autre ne récupère pas exactement les tuiles que le précédent vient de rendre.

Un `Joueur` a un nom, un score (mutable, `ajouterScore(int)` accepte les valeurs négatives pour les pénalités de fin de partie), et un `Chevalet`. Le `Chevalet` encapsule une `List<Tuile>` et expose principalement `retirerParId(String id)` — c'est cette méthode qui sera appelée à chaque pose pour transformer un `tileId` reçu de l'API en vraie tuile. Si l'ID n'existe pas, `retirerParId` retourne `null`, ce qui signale au contrôleur qu'il faut lever une erreur. Le chevalet sait aussi calculer ses `pointsRestants()`, utilisé pour les pénalités de fin de partie.

Une `Pose` (record immuable) représente une tuile en cours d'être posée à une position donnée, avec une `faceJoker` éventuelle. Ses méthodes `lettreVisible()` et `pointsLettre()` factorisent la logique « lettre effective vs valeur » : si la tuile est un joker, `lettreVisible()` retourne la face choisie en majuscule, et `pointsLettre()` retourne 0. Sinon, elles retournent les valeurs de la tuile. Un `Coup` agrège une liste de poses et une direction préférée (qui peut être `null` si on laisse le validateur la déduire).

Pourquoi des **records** pour `Position`, `Pose`, `Coup` (presque), `HistoriqueEntry`, `ResultatTour`, `ResultatValidation`, `InfoMot`, `PlacementCommande` ? Parce que ces classes sont des **valeurs** : elles n'ont pas d'identité, elles sont définies par leurs champs. Le record génère gratuitement `equals/hashCode/toString`, garantit l'immuabilité, et empêche d'oublier un setter. Pour le validateur, par exemple, pouvoir mettre des `Position` comme clés d'un `HashMap` (avec `equals` correct) est indispensable pour suivre les tuiles posées dans un coup.

6. Démarrer une partie

Quand le frontend, depuis l'écran de setup, envoie un `POST /api/game/start` avec `{"playerNames": ["Hugo", "Maria", "Yacine"]}`, voici ce qui se passe.

Le `JsonHandler` parse le corps en `Map`, et appelle la méthode `démarrer(...)` du serveur. Cette méthode extrait le tableau `playerNames` (via la méthode utilitaire `asStringList(...)` qui convertit n'importe quel `Object` en `List<String>`), puis appelle `contrôleur.nouvellePartie(noms)`.

Le contrôleur, lui, délègue à `partieService.démarrer(noms)`. Le service vérifie d'abord qu'il y a entre 2 et 4 noms ; sinon, `IllegalArgumentException("Il faut entre 2 et 4 joueurs.")` qui remontera comme HTTP 400. Pour chaque nom, il crée un `Joueur`. Il instancie une nouvelle `Partie` (qui à son tour crée un nouveau `Plateau` et un nouveau `SacTuiles` mélangé), puis appelle `partie.reinitialiserJoueurs(joueurs)` : cette méthode vide la liste interne, place les nouveaux joueurs, met `indexJoueurCourant = 0`, `passesConsecutifs = 0`, `terminee = false`, et **vide l'historique**. Pour chaque joueur, il appelle la méthode statique privée `completerChevalet(sac, joueur)` qui pioche tant que le chevalet n'a pas 7 tuiles ou que le sac n'est pas vide. Enfin, il pose un message du style « Partie créée. À Hugo de jouer. » dans `dernierMessage`, et retourne la `Partie`.

Retour côté contrôleur, qui stocke la partie dans son champ privé `partie`. Puis le serveur HTTP appelle `contrôleur.exporterEtat()` pour générer la réponse JSON.

`exporterEtat()` est la méthode la plus longue du contrôleur. Elle construit un `LinkedHashMap<String, Object>` avec toutes les clés que le frontend attend : `gameStarted`, `finished`, `currentPlayerIndex`, `currentPlayerName`, `bagCount`, `consecutivePasses`, `lastMessage`, `lastPoints`, `lastWords`, `canExchange` (vrai si `bagCount ≥ 7` et partie non terminée), `winner`, `players`, `rack`, `board`, `history`. Quand aucune partie n'existe, elle retourne un état « par défaut » avec un plateau vide (les 15×15 cases avec leurs primes mais sans tuiles), des joueurs vides, et `gameStarted: false`. C'est ce qui permet au frontend d'afficher le plateau dès l'écran d'accueil, avant même qu'une partie démarre.

Le `rack` exporté est **uniquement** celui du joueur courant. Les autres chevalets ne sont jamais sérialisés. C'est cohérent avec le mode multijoueur local : un seul écran, un seul joueur regarde, les autres ne doivent pas voir leurs tuiles. Les autres joueurs apparaissent dans `players` avec juste (`name`, `score`, `rackCount`, `current`) — on connaît la **taille** de leur chevalet, pas son contenu.

La sérialisation du plateau est une simple double boucle qui produit un tableau 2D d'objets (`row`, `col`, `prime`, `tile`). La prime est sortie en string (`MOT_TRIPLE`, `LETTRE_DOUBLE`, etc., via `prime.name()`), ce qui permet au frontend de générer dynamiquement la classe CSS `prime-mot_triple` en passant par `String(cell.prime).toLowerCase()`.

La sérialisation des joueurs et de l'historique suit le même principe : pour chaque entrée, un `LinkedHashMap` avec les bonnes clés. Pour l'historique, les clés sont `type`, `joueur`, `points`, `mots`, `message` — exactement les composants du record `HistoriqueEntry` (on y vient).

Le serveur emballe tout dans `Json.stringify(...)` et envoie la réponse en UTF-8 au frontend, qui la stocke dans son `gameState` et redessine le plateau, les chevalets, la sidebar.

7. Un tour de jeu de bout en bout

Voici le scénario : Hugo, joueur courant, sélectionne plusieurs tuiles et pose le mot `MAISON` à l'horizontale en passant par le centre. C'est le premier coup. On va suivre tout ce qui se passe.

7.1 Du clic à l'envoi de la requête

Côté frontend, l'écran de jeu affiche le plateau (`displayedBoard` — un memo qui fusionne la grille reçue du backend avec les `placements` locaux), la sidebar (joueurs, scores, dernière action, boutons d'action), et le chevalet (`availableRack` — le chevalet du joueur courant, **moins** les tuiles déjà posées localement).

Hugo clique sur la tuile `M` de son chevalet. La fonction `setSelectedTileId(tile.id)` retient l'ID. Visuellement, la tuile change de couleur (classe `selected`). Hugo clique alors sur la case (7,3) du plateau. La fonction `handleBoardCellClick(cell)` est appelée : elle vérifie que la case est libre (du backend) et qu'aucun placement local n'occupe déjà cette case ; si OK, elle appelle `placeTileOnBoard(selectedTileId, 7, 3)`. Cette fonction retrouve la tuile dans le rack via son ID, vérifie qu'elle n'est pas un joker (sinon elle ouvre un `window.prompt` pour la face), et ajoute un objet à `placements` : `{tileId: 'T12', row: 7, col: 3, jokerFace: '', letter: 'M', points: 3, joker: false}`. La sélection est réinitialisée.

Hugo continue : il clique sur `A`, puis sur la case (7,4), et ainsi de suite jusqu'à (7,8) avec le `N`. À ce stade, six tuiles sont posées localement (le `M` en (7,3), `A` en (7,4), `I` en (7,5), `S` en (7,6), `O` en (7,7), `N` en (7,8)). Aucune n'est encore envoyée au backend. C'est le rôle des `placements` locaux : permettre au joueur de poser, retirer, réorganiser ses tuiles librement avant de valider.

S'il s'aperçoit qu'il a fait une erreur, il peut cliquer sur la case posée pour annuler ce placement (le `handleBoardCellClick` retire l'entrée correspondante de `placements`), ou cliquer « Annuler les placements » pour tout remettre à zéro.

Quand il est satisfait, il clique « Valider le coup ». La fonction `handleValidateMove()` construit le payload :

```
{
  "direction": "HORIZONTALE",
  "placements": [
    {"tileId": "T12", "row": 7, "col": 3, "jokerFace": null},
    {"tileId": "T8", "row": 7, "col": 4, "jokerFace": null},
    {"tileId": "T19", "row": 7, "col": 5, "jokerFace": null},
    {"tileId": "T22", "row": 7, "col": 6, "jokerFace": null},
    {"tileId": "T3", "row": 7, "col": 7, "jokerFace": null},
    {"tileId": "T17", "row": 7, "col": 8, "jokerFace": null}
  ]
}
```



```
]
}
```

et appelle `playMove(payload)` du fichier `api.js`, qui fait un `POST /api/game/play`.

7.2 Le contrôleur déshabille le chevalier

Côté backend, le `JsonHandler` parse le corps. La méthode `jouer(...)` extrait la `direction` (en l'occurrence `HORIZONTALE`) et boucle sur le tableau `placements` pour construire une `List<PlacementCommande>`. Chaque `PlacementCommande` est un record (`tileId`, `ligne`, `colonne`, `faceJoker`) — c'est le seul rôle de cette classe : être l'objet de transport entre l'API et le contrôleur.

Le contrôleur reçoit donc une `List<PlacementCommande>` et la `Direction`. Sa méthode `jouerCoup(...)` fait un travail subtil. Elle commence par vérifier qu'une partie existe (sinon `IllegalStateException("Aucune partie en cours.")`) et que la liste n'est pas vide. Puis elle initialise deux listes : `prelevees` et `poses`.

Pour chaque `PlacementCommande`, elle doit transformer un `tileId` en vraie `Tuile`. Pour cela, elle appelle `joueur.getChevalier().retirerParId(placement.tileId())`. **Cette méthode retire physiquement la tuile du chevalier** — le chevalier est mutable, on en sort la tuile pour la mettre sur le plateau. Si l'ID n'existe pas, `retirerParId` retourne `null`, et le contrôleur lève `IllegalArgumentException("La tuile T12 n'existe plus dans le chevalier.")`. Sinon, la tuile est ajoutée à `prelevees` (au cas où il faudrait la remettre), et le contrôleur construit une `Pose(new Position(ligne, colonne), tuile, faceJoker)` qu'il met dans `poses`.

Une fois toutes les poses construites, le contrôleur appelle `partieService.jouerCoup(partie, new Coup(poses, direction))`. Mais **le tout est enveloppé dans un try/catch** : si jamais le service lève une exception (mot invalide, mot non contigu, hors plateau...), le contrôleur exécute `for (Tuile tuile : prelevees) joueur.getChevalier().ajouter(tuile)` pour **remettre toutes les tuiles dans le chevalier**, puis relance l'exception. Sans ce rollback, un coup rejeté ferait disparaître les tuiles définitivement. C'est un détail invisible mais essentiel.

7.3 Le validateur passe le coup au crible

`PartieService.jouerCoup(partie, coup)` commence par vérifier que la partie n'est pas terminée. Puis il appelle `validateur.valider(plateau, coup, partie.estPremierCoup())`. La méthode `partie.estPremierCoup()` délègue à `plateau.estVideCentre()` — vrai si la case (7,7) est encore vide. Dans notre scénario, c'est notre vrai premier coup : oui.

Le `ValideurCoup` est le composant le plus dense du projet. Il enchaîne une série de vérifications dans un ordre précis :

D'abord, **tuiles à l'intérieur du plateau et sur des cases vides**. La méthode boucle sur les poses et appelle `plateau.dansBornes(ligne, colonne)` puis `getCase(...).estVide()`. Elle construit en parallèle une `Map<Position, Pose> nouvellesPoses` qui sert à tout le reste de la validation. Si deux poses ont la même `Position`, le `put` retourne l'ancienne valeur, ce qui signale l'erreur « Deux tuiles sur la même case. ».

Ensuite, **les jokers ont une face**. Si `pose.tuile().estJoker()`, alors `pose.faceJoker()` doit être une lettre. Sinon, « Joker : choisis une lettre A-Z. ».

Vient la **déduction de la direction**. Quand 2+ tuiles sont posées, on regarde si elles partagent une ligne ou une colonne. Pour MAISON, toutes ont `ligne = 7` : direction horizontale. Si une seule tuile est posée, on regarde ses voisins déjà sur le plateau : un voisin gauche ou droite oriente vers l'horizontal, un voisin haut ou bas vers le vertical. Si les deux ou aucun ne sont pertinents, on prend la direction explicitement demandée par le client (`directionDemandee`), ou par défaut horizontale.

Ensuite, **alignement strict**. `posesAlignees(poses, direction)` vérifie que pour la direction horizontale, toutes les poses ont la même ligne (sinon erreur).

Puis, **contiguïté du mot principal**. `verifierContiguïte(...)` parcourt l'intervalle entre la pose minimale et la pose maximale dans la direction choisie, et vérifie que **chaque case intermédiaire** est soit une nouvelle pose, soit une tuile déjà sur le plateau. Sinon : « Mot non contigu. ». Dans notre cas, les six tuiles sont contiguës en (7,3) → (7,8), donc OK.

Puis, **règle du premier coup ou contact avec l'existant**. Comme c'est le premier coup, le validateur vérifie que `nouvellesPoses` contient bien la `Position(7, 7)` : on passe par le centre. OK. Si ce n'avait pas été le premier coup, il aurait à la place vérifié qu'au moins une nouvelle pose est adjacente (haut, bas, gauche, droite) à une tuile déjà sur le plateau ; sinon « Le coup doit toucher une tuile déjà sur le plateau. ».

Vient ensuite la **collecte des mots formés**. Pour le mot principal, le validateur construit le mot en partant de la première pose et en remontant dans la direction tant qu'il y a des lettres (poses ou tuiles existantes), puis en avançant lettre par lettre. Pour chaque lettre, il consulte la fonction utilitaire `lettreA(...)` qui regarde d'abord dans `nouvellesPoses` (et dans ce cas appelle `pose.lettreVisible()` pour gérer les jokers) puis dans le plateau. La `StringBuilder` accumule les caractères et la liste de positions est mémorisée, le tout étant agrégé dans un `InfoMot(texte, positions)`.

Pour chaque pose, le validateur construit aussi le mot perpendiculaire — donc, dans notre cas, le mot vertical qui passerait par chaque case posée. Comme c'est le premier coup et qu'il n'y a aucune autre tuile sur le plateau, ces mots perpendiculaires ne font qu'un caractère, et sont donc ignorés (longueur < 2). Le mot principal est MAISON, longueur 6.

Le validateur déduplique avec une `Set<String> signatures` basée sur la liste de positions (`infoMot.positions().toString()`) — ainsi un même mot ne peut pas être validé plusieurs fois s'il est trouvé par deux chemins différents.

Enfin, **vérification au dictionnaire**. Pour chaque mot (un seul ici, MAISON), le validateur appelle `normaliserMot(...)` (la même normalisation qu'au chargement du dictionnaire), puis `dictionnaire.estValide(motNormalise)`. Si non, `IllegalArgumentException("Mot invalide : MAISON")`. Si oui, on continue.

Le validateur retourne un `ResultatValidation(mots, motsNormalises)` : la liste des mots avec leurs positions (utilisée juste après pour le score) et la liste des mots normalisés (utilisée pour l'historique et la réponse JSON).

7.4 Le calcul du score

`PartieService.jouerCoup(...)` reçoit le `ResultatValidation` et appelle `serviceScore.calculer(plateau, coup, mots)`. Le calcul boucle sur les mots et, pour chaque mot, sur ses positions.

Pour chaque position du mot, le service distingue **nouvelle pose** (présente dans `nouvellesPoses`) ou **tuile existante**. Pour une nouvelle pose : il prend `pose.pointsLettre()` (qui retourne 0 si c'est un joker, sinon la valeur de la lettre), regarde la prime de la case via `plateau.getCase(...).getPrime()` et applique le multiplicateur **lettre** ($\times 2$ pour LD, $\times 3$ pour LT). Pour une **lettre déjà** sur le plateau : juste `tuile.getPoints()`, sans aucun multiplicateur — la prime a déjà été consommée au coup précédent.

En parallèle, pour chaque nouvelle pose, le service accumule un multiplicateur **mot** : $\times 2$ pour MD ou DEPART, $\times 3$ pour MT, neutre sinon. À la fin du mot, il multiplie la somme des valeurs lettres par ce multiplicateur mot global.

Pour notre **MAISON** posé en (7,3) à (7,8) : M (3 points) + A (1) + I (1) + S (1) + O (1) + N (1) = 8. La case (7,7) est DEPART, donc multiplicateur mot $\times 2 \rightarrow$ **16 points**.

Si le coup avait posé 7 tuiles d'un coup, le service aurait ajouté +50 pour le « scrabble ». Ici, seulement 6 tuiles : pas de bonus.

7.5 Application du coup et mise à jour de l'état

`PartieService.jouerCoup(...)` continue. Il appelle `plateau.appliquerCoup(coup)` qui boucle sur les poses et **pose chaque tuile** sur la `CasePlateau` correspondante via `poserTuile(tuile)`. Si une tuile était un joker, `Plateau.appliquerCoup(...)` la **remplace** par une nouvelle `Tuile(lettreVisible, 0)` : ainsi, sur le plateau, on n'a plus le ? ; on a la lettre choisie qui vaut 0 point. C'est ce qui « consomme » le joker.

Ensuite, le service ajoute les points au joueur : `joueur.ajouterScore(points)`. Hugo passe de 0 à 16. Puis `completerChevalet(sac, joueur)` repioche jusqu'à 7 tuiles tant que le sac n'est pas vide. Pour Hugo, qui en a posé 6, on en pioche 6 nouvelles. Le sac contient maintenant $102 - 6 \times 3$ (chevalets initiaux) - 6 (pioche après le coup) = 78 tuiles. (Maria et Yacine ont 21 tuiles à eux deux dans leur chevalet.)

`partie.setPassesConsecutifs(0)` parce qu'un coup posé remet le compteur à zéro.

Puis le service vérifie la **fin par chevalet vide** : si le sac est vide ET que le chevalet du joueur courant est aussi vide, c'est terminé. Pas notre cas ici. Si ça l'avait été, `appliquerFinDePartieChevaletVide(partie, joueur)` aurait fait : pour chaque joueur, soustraire ses points restants de son score, accumuler dans `sommeRestante`, et à la fin ajouter `sommeRestante` au score du joueur qui a vidé son chevalet — c'est le bonus de fin de partie.

Le service ajoute alors une **entrée d'historique** via

`partie.ajouterHistorique(HistoriqueEntry.coup(joueur.getNom(), points, mots))`. C'est le moment d'expliquer cette classe en détail.

7.6 Tenir l'historique des actions

`HistoriqueEntry` est un record immuable à cinq champs : `type` (chaîne "COUP", "PASSE", "ECHANGE" ou "FIN"), `joueur` (le nom — vide pour le type FIN), `points` (les points marqués — 0 pour PASSE/ECHANGE/FIN), `mots` (les mots formés — vide pour PASSE/ECHANGE/FIN), et `message` (une phrase préformatée prête à afficher).

Quatre fabriques statiques construisent les entrées avec le bon message selon le type. Pour notre coup : `HistoriqueEntry.coup("Hugo", 16, ["MAISON"])` produit l'entrée :

```
type      : "COUP"
joueur    : "Hugo"
points    : 16
mots      : ["MAISON"]
message   : "Hugo a joué pour 16 pts : MAISON"
```

Le message est calculé dans la fabrique : `joueur + " a joué pour " + points + " pts : " + String.join(", ", mots)`. La liste de mots est figée par `List.copyOf(...)`, ce qui garantit qu'on ne peut pas la modifier après coup.

Pourquoi ces choix ? D'abord, l'**immutabilité** : un événement passé n'a pas à être réécrit ; un record nous garantit que personne ne pourra modifier une entrée déjà ajoutée. Ensuite, le **message préformaté** : c'est le backend qui formate, parce que c'est lui qui connaît la grammaire (le `tuile(s)` de l'échange par exemple est dur à reproduire côté frontend). Le frontend n'a qu'à afficher `entry.message` — il ne reproduit aucune logique métier. Si demain on rajoute un client mobile ou une CLI, ils consomment exactement le même message.

L'historique est stocké dans `Partie.historique` (`List<HistoriqueEntry>`), et exposé en lecture seule via `getHistorique()` qui retourne un `Collections.unmodifiableList(...)`. On ne peut **qu'ajouter** des entrées, et seul le `PartieService` le fait :

- dans `jouerCoup(...)`, après que tout a réussi, une entrée `coup(...)` ;
- dans `passer(...)`, à chaque appel, une entrée `passe(...)` ;
- dans `echanger(...)`, à chaque appel, une entrée `echange(...)` ;
- dans les deux scénarios de fin de partie, en plus, une entrée `finPartie(message)`.

Le contrôleur sérialise chaque entrée en `LinkedHashMap` avec les clés `type`, `joueur`, `points`, `mots`, `message` (la méthode `serialiserHistorique(...)`). Cette liste est exposée sous la clé `history` de la réponse `/api/game/state`. Le frontend la trouve dans `gameState.history` et la parcourt à l'envers pour afficher les plus récentes en haut. Une classe CSS dynamique `history-type-coup` (ou `passe`, `echange`, `fin`) est ajoutée à chaque ligne pour la colorer différemment selon son type.

Les avantages de cette conception :

- **Auditabilité** : on peut rejouer mentalement la partie en lisant l'historique de bout en bout.
- **Affichage trivial** : `<span>{entry.message}</span>` suffit côté frontend.
- **Extensibilité** : ajouter un nouveau type (par exemple `ABANDON`) demande juste une nouvelle fabrique + une classe CSS, sans toucher aux services.
- **Robustesse** : impossible de corrompre l'historique depuis la couche supérieure ; impossible de modifier rétroactivement une entrée.

7.7 Préparer la suite et répondre

Après l'ajout à l'historique, le service met à jour `dernierMessage` (« Coup validé pour 16 points. »), `derniersPoints` (16), `derniersMots` (`["MAISON"]`). Ces trois champs sont lus par le contrôleur pour la sortie JSON (`lastMessage`, `lastPoints`, `lastWords`) et affichés par le frontend dans la boîte « Dernière action ».

Si la partie n'est pas terminée, le service appelle `partie.passerAuJoueurSuivant()`, qui incrémente l'index modulo le nombre de joueurs. C'est maintenant à Maria.

Le service retourne un `ResultatTour(16, ["MAISON"], "Coup validé pour 16 points.", false)` au contrôleur, qui le stocke dans une variable `resultat`. Le serveur HTTP construit la réponse en appelant à nouveau `controleur.exporterEtat()` pour avoir l'état complet, puis ajoute une clé `action` qui contient les détails du coup qui vient d'être joué : `{points: 16, words: ["MAISON"], message: "...", finished: false}`. Cela permet au frontend d'afficher un toast ou un effet visuel.

Côté frontend, la fonction `handleValidateMove(...)` reçoit la réponse, met à jour `gameState`, vide `placements` (les placements locaux sont maintenant officiels, plus besoin de les mémoriser), et déclenche `setTurnTransition(true)` parce que la partie continue. L'overlay « C'est au tour de Maria — passe l'appareil » apparaît, masquant le chevalet d'Hugo. Quand Maria clique « Voir mon chevalet », le `turnTransition` repasse à `false` et son chevalet (récupéré via `gameState.rack`) s'affiche.

8. Quand le coup est rejeté

Si Hugo avait posé un mot invalide — par exemple `MAOSI` au lieu de `MAISON` — la chaîne de validation aurait échoué à l'étape « validité au dictionnaire ». Le validateur aurait levé `IllegalArgumentException("Mot invalide : MAOSI")`.

Le `PartieService.jouerCoup(...)` ne capture pas cette exception : il la laisse remonter au contrôleur. Le contrôleur, lui, la **capture** dans son `try/catch` et exécute le rollback : pour chaque tuile dans `prelevees`, il appelle `joueur.getChevalet().ajouter(tuile)`. Les tuiles reviennent dans le chevalet exactement comme avant (l'ordre peut différer mais peu importe — les IDs restent les mêmes). Puis le contrôleur relance l'exception.

Le `JsonHandler` du serveur capture l'`IllegalArgumentException` et envoie un HTTP 400 avec `{"error": "Mot invalide : MAOSI"}`. Côté frontend, la fonction `handleValidateMove` attrape l'erreur dans son `catch (err)` et appelle `setError(err.message)`. Le bandeau rouge en haut de page affiche le message. **Les placements locaux ne sont pas vidés** : le joueur peut corriger sa pose sans tout refaire.

C'est exactement le comportement qu'on attend : pas de tuile perdue, pas d'état incohérent, message clair.

9. Échanger des tuiles, passer son tour

L'échange de tuiles suit le même schéma global. Le frontend envoie `{tileIds: ["T8", "T22"]}` à `/api/game/exchange`. Le contrôleur appelle `partieService.echanger(partie, idsTuiles)`.

Le service vérifie d'abord trois choses : que la liste n'est pas vide (sinon « Choisis au moins une tuile à échanger. »), que le sac contient au moins 7 tuiles (« Le sac doit contenir au moins 7 tuiles pour échanger. »), et que le sac contient au moins autant de tuiles que ce qu'on veut échanger.

Pour chaque ID, il appelle `joueur.getChevalet().retirerParId(id)`, et stocke les tuiles retirées dans `retirees`. **Comme pour la pose**, si une tuile est introuvable, il restaure les tuiles déjà retirées et lance l'exception.

Ensuite, il pioche `n` nouvelles tuiles via `partie.getSac().piocher(idsTuiles.size())` et les ajoute au chevalet. Puis `partie.getSac().remettre(retirees)` rend les anciennes tuiles au sac et re-mélange tout — pour empêcher qu'un joueur qui échangerait juste après ne récupère exactement ce que le précédent a rendu.

Une entrée d'historique `HistoriqueEntry.echange(joueur.getNom(), idsTuiles.size())` est ajoutée. Le compteur de passes consécutives est incrémenté (un échange « consomme » un tour sans coup posé), et si le seuil `2 × nombre de joueurs` est atteint, la partie se termine.

Passer un tour est encore plus simple : `partieService.passer(partie)` incrémente le compteur, ajoute une entrée `HistoriqueEntry.passe(joueur)`, vérifie le seuil, et passe la main.

10. La fin de partie

Deux scénarios mettent fin à la partie. Dans les deux, c'est `PartieService` qui détecte la condition et applique les règles.

Fin par chevalet vide : à la fin de `jouerCoup(...)`, après avoir reconstitué le chevalet (ce qui ne fait rien si le sac était déjà vide), le service teste `partie.getSac().estVide() && joueur.getChevalet().taille() == 0`. Si vrai, il appelle `appliquerFinDePartieChevaletVide(partie, joueur)`. Cette méthode boucle sur tous les joueurs, calcule pour chacun `joueur.getChevalet().pointsRestants()`, soustrait cette somme de leur score, et accumule les pénalités dans `sommeRestante`. À la fin, elle ajoute `sommeRestante` au score du `gagnantPotentiel` (le joueur qui vient de finir). C'est le bonus standard du Scrabble. Puis `partie.setTerminee(true)`, message « Fin : un joueur a vidé son chevalet et le sac est vide. », et une entrée `HistoriqueEntry.finPartie(message)`.

Fin par excès de passes : dans `passer(...)` et `echanger(...)`, après avoir incrémenté `passesConsecutifs`, si la valeur atteint `2 × nombre de joueurs`, le service appelle `appliquerFinDePartieParPasses(partie)` qui pénalise tous les joueurs **sans bonus** (chacun perd ses points restants, point final). Puis `setTerminee(true)`, message « Fin : trop de passes consécutives. », entrée d'historique `finPartie(...)`.

Une fois `terminee = true`, l'export d'état renseigne `winner` via `partie.getNomGagnant()` qui parcourt les joueurs et retourne le nom de celui au plus haut score. Le frontend détecte `gameState.finished` et affiche `GameOverScreen` : un classement décroissant, un récapitulatif complet de l'historique, et un bouton « Nouvelle partie » qui appelle `resetGame()` (POST `/api/game/reset`). Côté backend, `reinitialiser()` met simplement `partie = null` ; la prochaine requête `/api/game/state` retournera l'état « par défaut ».

11. Le dictionnaire en pratique

Le dictionnaire est un fichier texte (`dictionnaire/mots.txt`), un mot par ligne, encodage UTF-8. La normalisation appliquée au chargement est la même que celle appliquée au moment de la validation : `île`, `Ile`, `IlE` deviennent tous `ILE`. Cela permet d'utiliser un fichier source qui peut contenir des accents, sans avoir à les retirer manuellement.

Le service implémente l'interface `ServiceDictionnaire` à une seule méthode (`estValide`). Cette interface a trois implémentations : `ServiceDictionnaireFichier` (production, charge depuis disque), `ServiceDictionnaireMemoire` (utile pour les tests, reçoit les mots à la construction), et

`ServiceDictionnaireToujoursValide` (fallback / développement). Le validateur ne sait pas laquelle on lui injecte — il appelle juste `estValide(mot)`. C'est ce qui permet de basculer entre les trois sans toucher au validateur.

Pourquoi un fichier local et pas une API distante ? Trois raisons. La performance d'abord : la validation doit être instantanée. Une API distante ajouterait des centaines de millisecondes par coup et nous rendrait dépendants d'un service tiers. Le hors-ligne ensuite : le projet doit tourner sans connexion. L'indépendance enfin : pas de service externe qui peut disparaître ou changer ses conditions d'utilisation.

Pour une partie homologuée, le fichier livré ne suffit pas (ODS contient $\approx 380\,000$ mots). Les scripts `dictionnaire/telecharger.bat` (Windows, via PowerShell) et `dictionnaire/telecharger.sh` (Unix, via curl) téléchargent une liste publique de mots français depuis un dépôt GitHub. Le fichier obtenu (`mots_complet.txt`) peut remplacer `mots.txt`. Le format est identique, la normalisation aussi — il n'y a rien d'autre à faire que renommer le fichier.

12. Le frontend dans tout ça

Côté frontend, le fichier central est `App.jsx`. Il porte tout l'état UI dans une dizaine de `useState` :

- `gameState` est la dernière snapshot reçue du backend — la **source de vérité** pour le rendu (plateau, joueurs, historique). Tout ce que le frontend affiche en sort, sauf les placements en cours.
- `placements` est la liste des tuiles déposées localement avant validation. Cette dualité « état distant + état local » est ce qui permet au joueur de poser plusieurs tuiles avant de soumettre.
- `selectedTileId`, `direction`, `exchangeMode`, `selectedExchangeIds` gèrent les interactions.
- `turnTransition` déclenche l'overlay entre joueurs.

Deux `useMemo` font le pont entre l'état distant et local : `availableRack` retire du chevalet visible les tuiles posées localement, et `displayedBoard` fusionne la grille reçue du backend avec les `placements` (les cases concernées prennent la classe `pending` et affichent la lettre choisie).

La pose d'une tuile se fait par clic ou drag-and-drop. Les deux passent par la même fonction `placeTileOnBoard(tileId, row, col)`, qui vérifie disponibilité de la case, demande la face si c'est un joker, et ajoute l'entrée à `placements`. Cette unification garantit que les deux modes ont exactement le même comportement.

Quand le joueur valide, `handleValidateMove()` envoie le payload complet et, sur succès, vide `placements` et déclenche la transition. Sur erreur, il garde les placements et affiche le message d'erreur — cohérent avec le rollback côté backend.

Le fichier `api.js` est volontairement court : une fonction `request(path, options)` qui factorise URL de base, en-tête `Content-Type`, parsing JSON et conversion d'erreur ; et six fonctions exportées qui appellent chacune un endpoint. Aucune logique métier — juste du transport.

`styles.css` contient l'apparence. Le plateau utilise `display: grid; grid-template-columns: repeat(15, 1fr); grid-template-rows: repeat(15, 1fr); aspect-ratio: 1/1` pour rester carré, et `width: min(100%, calc(100vh - 290px))` pour s'adapter à la hauteur viewport — c'est ce qui garantit que tout tient à 100 % de zoom sur un écran standard, sans débordement horizontal.

13. Lancement et build

Le backend se compile et se lance via `run-backend.bat` (Windows) ou `run-backend.sh` (Unix). Ces scripts génèrent une `sources.list` avec tous les `.java` sous `src/`, appellent `javac -encoding UTF-8 -d out @sources.list`, puis lancent `java -cp out view.Main`. Aucune dépendance externe : un JDK 17+ suffit. La compilation est instantanée parce qu'il n'y a rien à télécharger.

Le frontend se lance avec Vite : `cd frontend-react && npm install` (la première fois) puis `npm run dev` (serveur de dev avec hot-reload sur `http://localhost:5173`). Pour la prod, `npm run build` produit un bundle dans `frontend-react/dist/`.

Pour développer : un terminal lance le backend, un autre lance le frontend, et on ouvre `http://localhost:5173`. Le CORS est ouvert côté backend (`Access-Control-Allow-Origin: *`), donc les deux ports communiquent sans configuration supplémentaire. Sous PowerShell, il faut préfixer le script Windows par `.\` (`.\run-backend.bat`), car PowerShell ne lance pas les scripts du dossier courant par défaut.

14. Pourquoi ces choix techniques

Pas de framework HTTP côté Java. Le `com.sun.net.httpserver.HttpServer` est dans le JDK depuis Java 6. Pour notre besoin (sept endpoints, pas de middleware complexe), il est parfaitement suffisant. On évite Spring, Javalin et leurs centaines de Mo de dépendances. Conséquence : démarrage instantané, build trivial.

Pas de bibliothèque JSON. `Json.java` (≈ 250 lignes) couvre nos besoins exactement. Avantages : zéro dépendance, projet reproductible avec un simple `javac`. C'est délibérément minimal — pas de support des annotations, pas de Jackson-style customization. Si on devait scaler, on remplacerait par Jackson, mais à notre échelle ça vaut le coup.

État sur le serveur, frontend stateless (presque). Tout l'état métier vit côté backend. Le frontend ne stocke que des intentions transitoires. Si l'utilisateur recharge, le frontend redemande l'état et reprend à l'identique. Cela élimine toute classe entière de bugs de désynchronisation.

Validation centralisée. Toutes les règles sont dans `ValideurCoup` et `ServiceScore`. Le frontend n'a aucune logique métier. Conséquence : on ne peut pas tricher en bidouillant le client, et l'expérience reste cohérente même si on ajoute un autre client (mobile, CLI).

Tuiles avec ID stable. Sans cet ID, on ne pourrait pas distinguer « cette tuile-ci » de « celle-là » quand elles ont la même lettre. C'est ce qui rend l'API REST si simple : un placement = `(tileId, row, col)`, et le contrôleur se débrouille pour retrouver la tuile.

Records partout où c'est possible. Les valeurs (positions, poses, coups, événements d'historique, résultats) sont des records. Cela donne `equals/hashCode/toString` gratuits, garantit l'immuabilité, et évite la moindre erreur d'oubli de `setter`.

Historique préformaté côté backend. Le message de chaque entrée d'historique est calculé dans la fabrique `HistoriqueEntry.coup/passe/echange/finPartie`. Le frontend n'a qu'à afficher la chaîne. Si demain un autre client se branche, il consomme exactement le même format.

Couches strictes. `model` ne dépend que de Java. `service` ne dépend que de `model`. `controller` dépend de `model` + `service`. `api` dépend de tout sauf de `view`. Tester `ValideurCoup` ou `ServiceScore` ne nécessite

pas de serveur HTTP. Remplacer `api` par une CLI ne demande pas de toucher au reste.

15. Limites connues et pistes d'évolution

Le dictionnaire livré est une base courante de mots français — suffisant pour démontrer la mécanique, mais pas pour une partie homologuée. Les scripts de téléchargement aident, mais l'idéal serait d'utiliser une liste type ODS ($\approx 380\,000$ mots).

Il n'y a pas de persistance : redémarrer le backend efface la partie en cours. Brancher une persistance serait simple — la `Partie` est sérialisable sans difficulté (tous ses champs sont soit primitifs, soit des collections de records ou de classes simples). Un `dump JSON` à chaque coup et un `load` au démarrage suffiraient.

Le mode multijoueur est local : tout passe par un seul navigateur. Pour un client par joueur, il faudrait introduire des sessions côté API, et un canal de push (WebSocket ou Server-Sent Events) pour synchroniser plusieurs clients en temps réel.

Pas de tests automatisés. C'est le premier ajout à faire : tester `ValideurCoup` et `ServiceScore` avec des plateaux fixés est trivial et rassurant. Une suite JUnit minimale ferait des merveilles.

Les messages sont en français en dur dans le code. Pour internationaliser, il faudrait extraire les chaînes des entrées d'historique et des messages d'erreur, et gérer un fichier de ressources par langue.